EDA (Introduction)

What is EDA(Event Driven Architecture):

Considered it as a system design style where:

- Services do not call each other directly.
- Instead, Services emit events and other services react to those events.

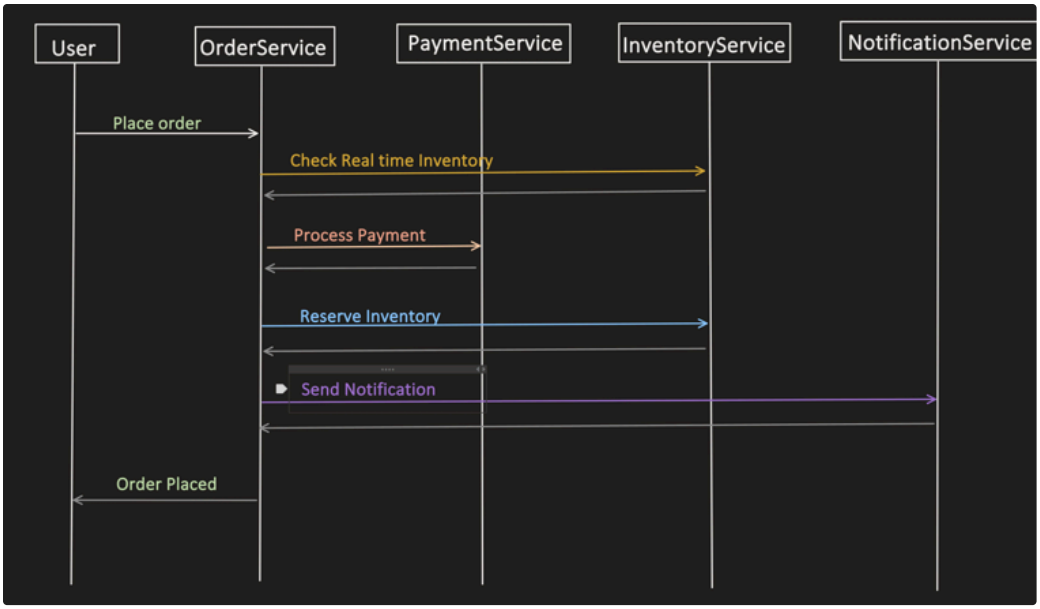
What is Event:

A fact that something happened in the past.

Its properties:

Property	Description	Example
Immutable	Can not be changed once created	OrderCreated at 10:30PM
Past Tense	Describes what happened, not what to do	OrderCreated, Not Place Order
Self-Contained	Has all info needed to understand it.	Contains orderId, amount, timestamp.

In REST Distributed Microservices:



Problems with Synchronous Chain:

1. **Availability:**
   All services must be available at the same time. Any Service down, request will fail.

2. **Latency Accumulation:**
   Total Latency = Sum of all service latencies.

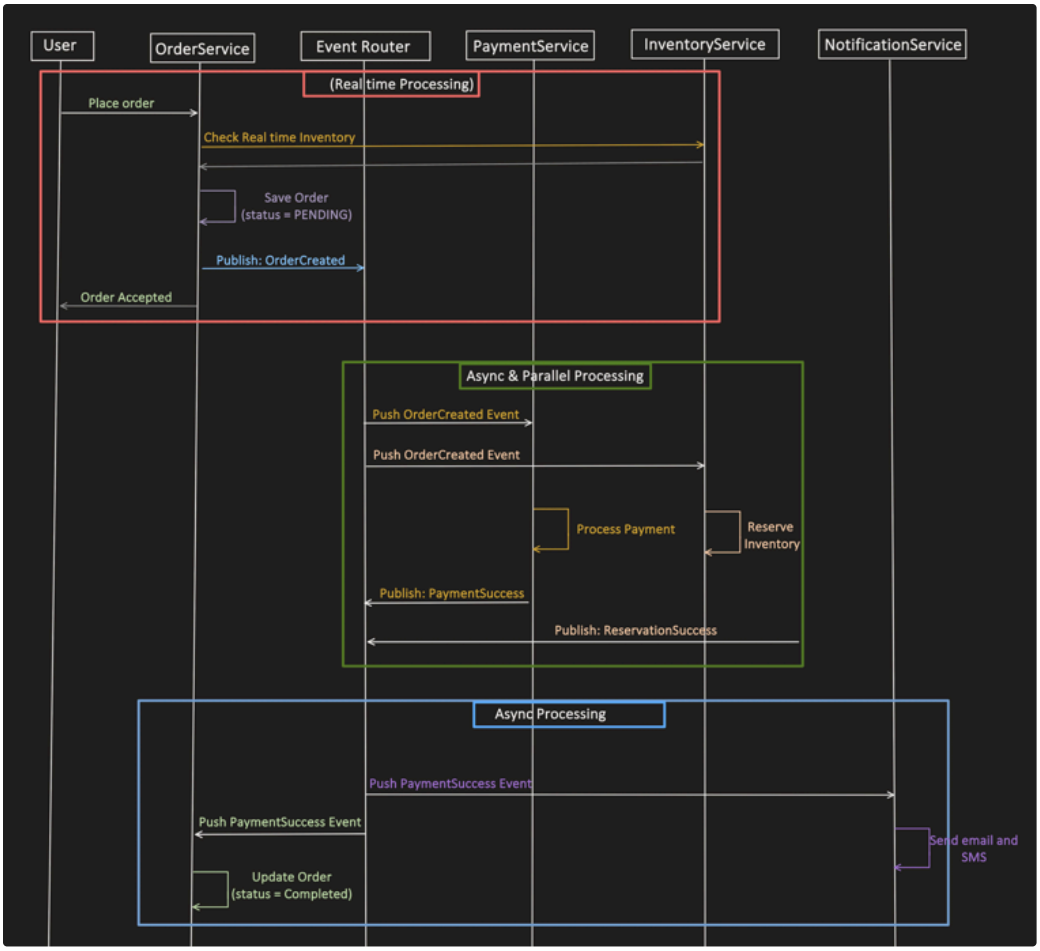
3. **Cascading Failure:**
   One slow service can bring down other services and entire flow.

4. **Tight Coupling:**
   In above example, Order Service knows about Payment, Inventory and Notification service. Their endpoint, how to invoke, their response structure etc.

5. **Scaling issue:**
   We can not scale one service independently based on load.

In Event Driven Architecture (EDA) Distributed Microservices:

Note: Its just a example to understand EDA



Advantages of Event Driven Architecture:

1. Loose Couple:
   No direct REST dependency.

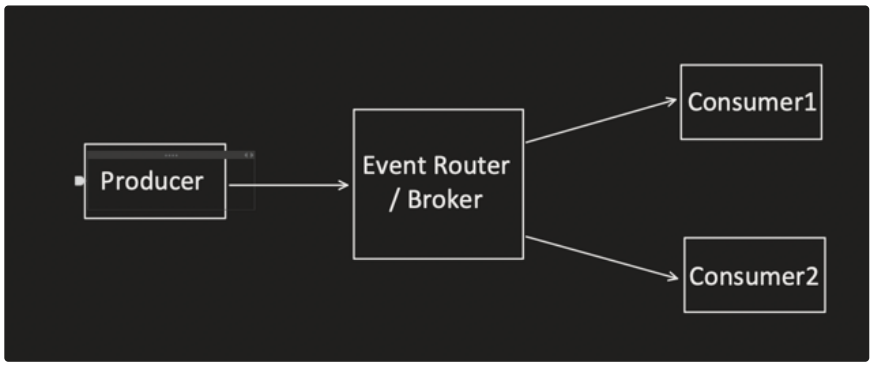
2. Better Scalability:
   Services can scale independently.

3. Better Resilience:
   Temporary issue do not break the whole system.

4. Replay:
   We can re-process old events.

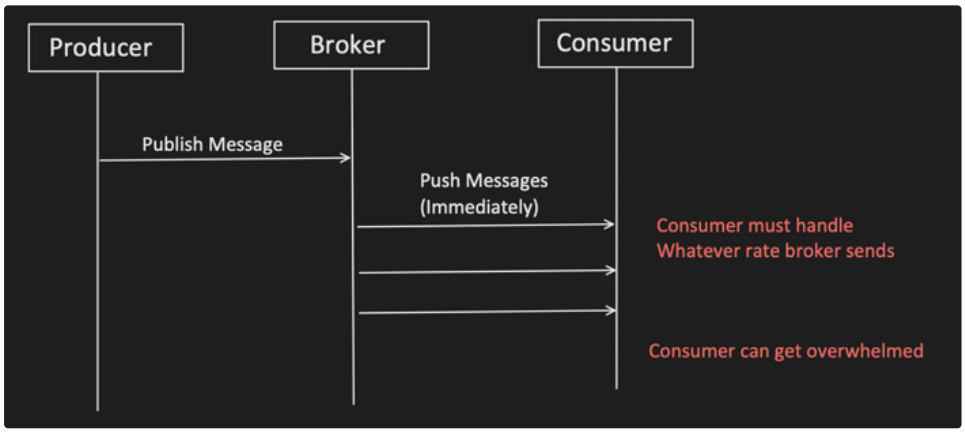
5. Improvement in latency.

So from above understanding, key Event Driven Architecture components are:

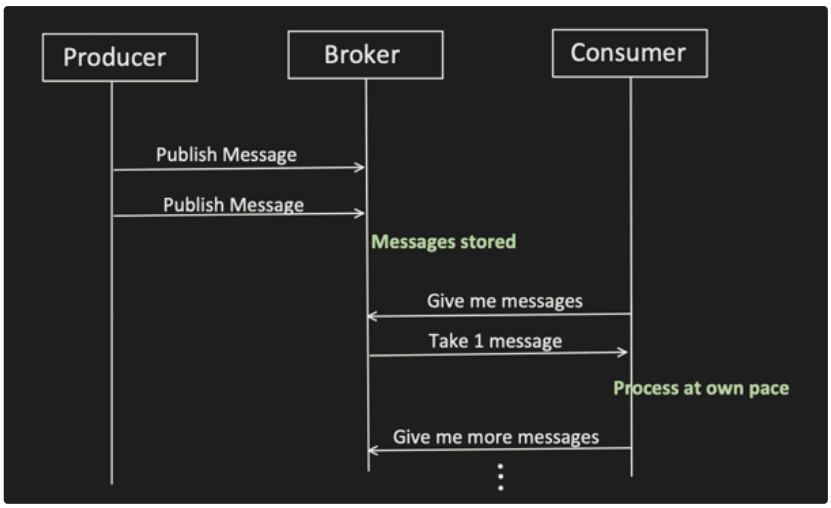


How do events move through EDA:

Push:



Pull:



EDA Models: Pub/Sub vs Streaming

Pub/Sub:

- Events are published to active consumers and then forgotten.
- If new consumer joined say tomorrow, it wont get yesterday messages.

Used when:

- We care about just delivery.
- Not about storing history.

Example:

- RabbitMQ Exchange

Streaming:

- Events are appended in logs forever or retention based.

Used when:

- We want replay.
- We want multiple consumer

If new consumer joined say tomorrow, it can read from latest offset or read from day1 (if retention allows)

Example:

- Kafka

Challenges of Event Driven Architecture:

1. **Eventual Consistency:**

As events are processed in Async manner. Get call might return stale data, but sub-sequent calls might give up-to-date information.

2. **Duplicate events:**
   Most event router guarantee "at-least once delivery". So duplicates are normal.

3. **Ordering problems:**
   Events might reached in out-of-order.

4. **Schema evolution:**

   If schema breaks, all consumer will crash.

5. **Debugging complexity:**

Distributed tracing across async hops are little difficult.

6. **Poison messages:**

1 bad message can block the Event router.

7. **Operational overhead:**

Must monitor for:

- Consumer lag
- Throughput
- Partition, queue etc.

<u>Use-cases of Event Driven Architecture, When to go for it:</u>

1. **1 event → N consumer:**

When one action must trigger multiple independent actions.

Ex:

OrderPlaced, then:

- Inventory reserves
- Payment charges
- Email sent
- Loyalty points added
- Analytics updated

2. **Long running business workflow:**

Ex:
order → payment → shipment → delivery

These flows take time and can fail midway. So flow is not in critical path, it can be broken into multi-step which is failure prone.

3. **Eventual consistency is acceptable.**

4. **Real time analytics:**

Data need to be continuously processed.